EXPERIMENT NO. -5

Aim

To create SQL views for department salary summary and employee hierarchy, test the updatability of views, and implement a recursive CTE to display reporting chains.

```sql
ALTER TABLE Employee
ADD ManagerID INT;
UPDATE Employee
SET ManagerID = NULL
WHERE EmpID = 1;

UPDATE Employee
SET ManagerID = 1
WHERE EmpID IN (2,3,4,5,6);

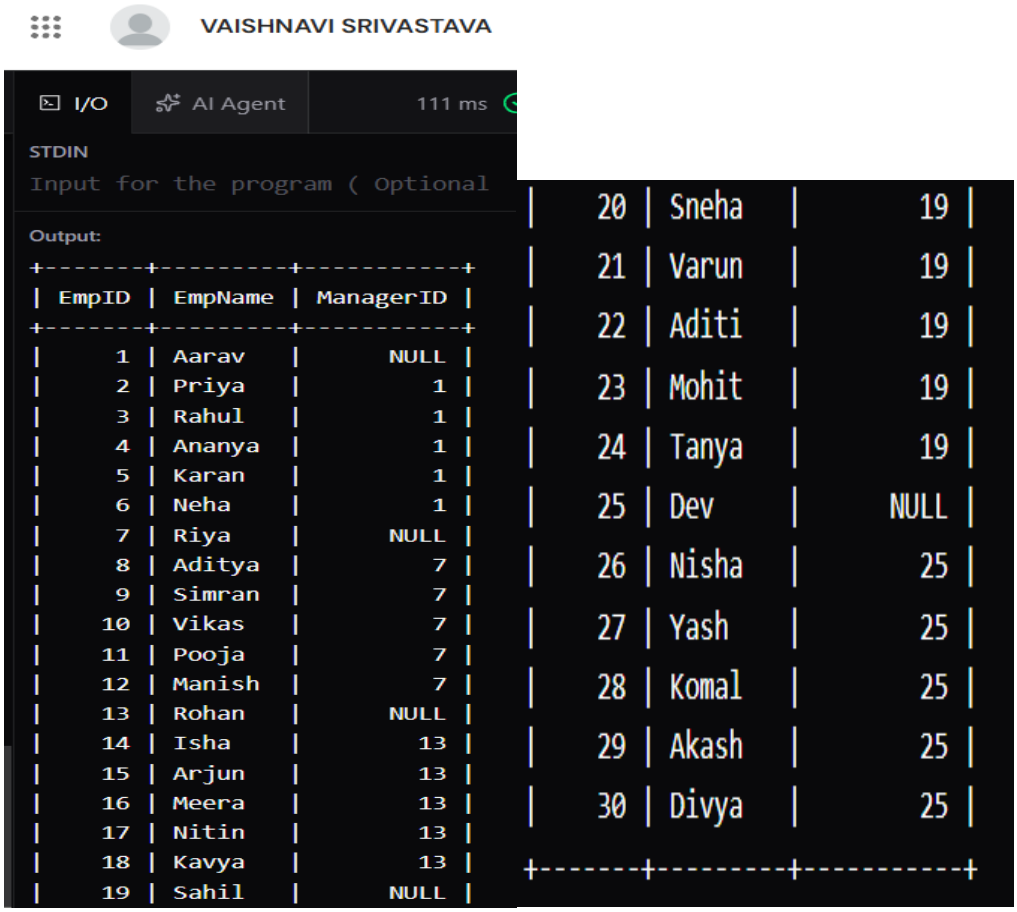
UPDATE Employee
SET ManagerID = 7
WHERE EmpID IN (8,9,10,11,12);

UPDATE Employee
SET ManagerID = 13
WHERE EmpID IN (14,15,16,17,18);

UPDATE Employee
SET ManagerID = 19
WHERE EmpID IN (20,21,22,23,24);

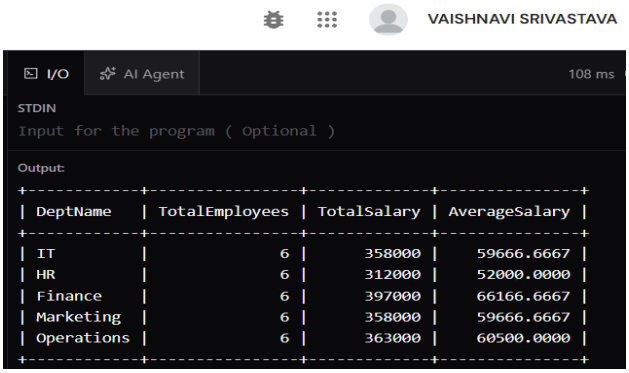
UPDATE Employee
SET ManagerID = 25
WHERE EmpID IN (26,27,28,29,30);
SELECT EmpID, EmpName, ManagerID
FROM Employee;
```

EXPECTED OUTPUT:-



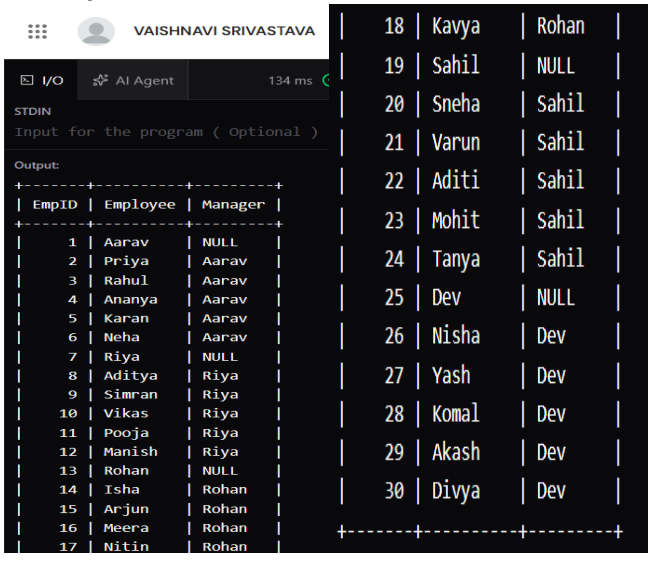
2.Create Department Salary Summary View

```
CREATE VIEW DepartmentSalarySummary AS
SELECT
D.DeptName
COUNT(E.EmpID) AS TotalEmployees
SUM(E.Salary) AS TotalSalary,
 AVG(E.Salary) AS AverageSalary
FROM Department D
JOIN Employee E
ON D.DeptID = E.DeptID
GROUP BY D.DeptID, D.DeptName;
SELECT * FROM DepartmentSalarySummary;
```



3.Create Employee Hierarchy View

```
CREATE VIEW EmployeeHierarchy AS
SELECT
  E.EmpID,
  E.EmpName AS Employee,
  M.EmpName AS Manager
FROM Employee E
LEFT JOIN Employee M
ON E.ManagerID = M.EmpID;
SELECT * FROM EmployeeHierarchy;
```



4.Test Updatability of Department Salary View
Query:

```
UPDATE DepartmentSalarySummary
SET TotalSalary = 500000
WHERE DeptName = 'IT';
```

Expected Result
Error / View is not updatable
Reason:
In DepartmentSalarySummary view :
COUNT()
SUM()
AVG()
GROUP BY
Has been used
This being a reason view is not directly updated

The DepartmentSalarySummary view is not updatable because it contains aggregate functions and GROUP BY.

---

6. Test Updatability of Employee Hierarchy View
Query:

```
UPDATE EmployeeHierarchy
SET Employee = 'Aarav Sharma'
WHERE EmpID = 1;
```

Expected Result:
Error / View is not directly updatable
Reason:
In EmployeeHierarchy view **self-join** has been used.

The EmployeeHierarchy view is not directly updatable because it is based on a selfjoin.

Recursive CTE to Display Employee Reporting Chains:

A **Recursive CTE** is a temporary result set that refers to itself. It is useful for displaying hierarchical data. In this practical:

- ManagerID stores the manager of an employee.
- Employees with ManagerID = NULL are treated as top-level employees.
- The recursive part finds employees who report to those managers.
- The Level column shows the hierarchy level.

RECURSIVE QUERY:-

```sql
WITH RECURSIVE EmployeeChain AS
(
    -- Anchor Query: Find top-level employees
    SELECT
        EmpID,
        EmpName,
        ManagerID,
        1 AS Level
    FROM Employee
    WHERE ManagerID IS NULL

    UNION ALL

    -- Recursive Query: Find employees reporting to each manager
    SELECT
        E.EmpID,
        E.EmpName,
        E.ManagerID,
        EC.Level + 1
    FROM Employee E
    INNER JOIN EmployeeChain EC
        ON E.ManagerID = EC.EmpID
)

SELECT
    EmpID,
    EmpName,
    ManagerID,
    Level
FROM EmployeeChain
ORDER BY Level, EmpID;
```

EXPECTED OUTPUT;-

VAISHNAVI SRIVASTAVA

I/O | AI Agent | 105 ms

STDIN

Input for the program (Optional)

Output:

EmpID	EmpName	ManagerID	Level
1	Aarav	NULL	1
7	Riya	NULL	1
13	Rohan	NULL	1
19	Sahil	NULL	1
25	Dev	NULL	1
2	Priya	1	2
3	Rahul	1	2
4	Ananya	1	2
5	Karan	1	2
6	Neha	1	2
8	Aditya	7	2
9	Simran	7	2
10	Vikas	7	2
11	Pooja	7	2
12	Manish	7	2
14	Isha	13	2
15	Arjun	13	2
16	Meera	13	2
17	Nitin	13	2
18	Kavya	13	2
20	Sneha	19	2
21	Varun	19	2
22	Aditi	19	2
23	Mohit	19	2
24	Tanya	19	2
26	Nisha	25	2
27	Yash	25	2
28	Komal	25	2
29	Akash	25	2
30	Divya	25	2

Result

Thus, SQL views for department salary summary and employee hierarchy were successfully created and tested. A recursive CTE was also successfully implemented to display employee reporting chains.